3 EF Core əlaqəli məlumatı necə gətirir

Kafedə sifarişlər ayrıca cədvəldə, sifarişə daxil olan məhsullar isə başqa cədvəldə saxlanır. **Cədvəl** eyni quruluşlu qeydlər toplusudur. Hər qeyd bir **sətir**, qeydin adı və məbləği kimi hissələri isə **sütun** adlanır.

Orders cədvəli	Müştəri	Məbləğ
42 nömrəli sifariş	7	30
43 nömrəli sifariş	7	10
44 nömrəli sifariş	9	20

42 nömrəli sifarişin daxilində çay və tort var. Bunlar OrderItems cədvəlində ayrı sətirlərdir. Hər item sətrində `OrderId = 42` yazılır ki, hansı sifarişə aid olduğu bilinsin. `Id` sifarişi fərqləndirən **əsas açar**, `OrderId` isə əlaqəni saxlayan **xarici açardır**.

EF Core baza məlumatı ilə C# obyektləri arasında işləməyi asanlaşdıran kitabxanadır. Bazada sifariş sətri, proqramda isə `Order` obyekti olur. Belə uyğunlaşdırma edən vasitələrə **ORM** deyilir.

Bu dərsin sualı budur: **sifarişi gətirəndə onun içindəki məhsulları da nə vaxt gətirməliyik?**

Order ilə order.Items arasındakı fərq

`Order` obyektinin `Id` və `Total` kimi adi məlumatları ola bilər. `Items` isə həmin sifarişin məhsullarına keçid verir.

C# ifadələrinin mənası

```
order.Id       // Sifarisin nomresi.
order.Total    // Sifarisin meblegi.
order.Items    // Sifarise daxil olan setirler.
```

Başqa əlaqəli obyektə və ya obyektlər siyahısına keçid verən xassəyə **navigation property** deyilir. `Items` çoxlu elementə, `Customer` isə bir müştəriyə keçid ola bilər. Xassə obyektin oxuya bildiyimiz və bəzən dəyişə bildiyimiz məlumat hissəsidir.

DbContext baza sorğularını göndərmək və lazım gəlsə obyekt dəyişikliklərini izləmək üçün istifadə etdiyimiz EF Core obyektidir. Nümunələrdə ona `db` deyəcəyik.

C#

```
var query = db.Orders.Where(o => o.Id == 42);
var order = await query.SingleAsync();
```

Birinci sətir sorğunu qurur. `Where` uyğun sətirləri seçən şərtdir. İkinci sətir onu icra edib bir nəticə istəyir. `SingleAsync` dəqiq bir nəticə gözləyir; sıfır və ya bir neçə nəticə xətadır. `await` nəticə hazır olana qədər asinxron gözləməyə imkan verir; bunu ayrıca async dərsində açacağıq.

Vacib nəticə: Order-un alınması Items-in də alındığını avtomatik sübut etmir.

Eager loading lazım olan əlaqəni əvvəlcədən seçir

Müştəri “sifarişin detalları” ekranını açır. Bilirik ki, sifarişlə yanaşı içindəki məhsullar da lazımdır. Ona görə hər ikisini əvvəlcədən tələb edirik.

C#

```
var order = await db.Orders
    .Include(o => o.Items)
    .SingleAsync(o => o.Id == 42);
```

`Include(o => o.Items)` “bu sifarişin Items əlaqəsini də gətir” deməkdir. Bu üsul **eager loading** adlanır. Məna etibarilə əlaqəli məlumatı əsas nəticə alınarkən hazır edirik.

İcra tamamlanandan sonra proqramda bu məlumat mövcuddur:

Nəticə

```
Order 42
    Items
        Cay
        Tort
```

İndi Items siyahısını gəzmək üçün həmin əlaqəni yenidən bazadan istəmək lazım deyil.

Əlaqənin davamını da seçmək olar. Məsələn, hər item-in Product məhsul məlumatı lazımdırsa `ThenInclude` həmin növbəti əlaqəni bildirir.

Eager loading-in mənası **əlaqəni əvvəlcədən istəməkdir**. “Həmişə bir SQL əmri olur” onun tərifi deyil. Sorğunun bir və ya bir neçə SQL əmrinə bölünməsinə bu dərsin sonrakı hissəsində baxacağıq.

Seçim: məlumatın lazım olacağını əvvəlcədən bilirsənsə eager loading aydın başlanğıcdır.

Explicit loading sonradan açıq əmr verir

İndi başqa ekran düşün. Əvvəl yalnız sifariş nömrəsi və məbləği görünür. Məhsullar istifadəçi “detalları aç” deyəndə lazımdır.

Əvvəl əsas sifarişi alırıq. Sonra ayrıca əmrlə Items-i yükləyirik:

C#

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
```

`Entry(order)` həmin obyektin EF Core məlumatına çıxış verir. `Collection` çoxlu element saxlayan əlaqəni seçir. `LoadAsync` onu yükləyir. Tək müştəri əlaqəsi üçün `Collection` yerinə `Reference` istifadə olunur.

Bu üsul **explicit loading** adlanır. Explicit “açıq şəkildə göstərilmiş” deməkdir. Əlaqənin hansı sətirdə yükləndiyini koddan görə bilirik.

Bu sadə nümunədə birinci sifariş sorğusu, sonra məhsullar üçün əlavə sorğu işləyir. Üstünlük ehtiyac yarananda yükləməkdir. Xərc isə əlavə baza müraciətidir.

Burada yüklənmiş Order və həmin Order-la işləyən DbContext hələ istifadə edilə bilən vəziyyətdə olmalıdır. Context-in işini bitirib bağladıqdan sonra onunla yeni sorğu göndərə bilmərik.

Seçim: əlaqə bəzən lazımdırsa və əlavə müraciəti açıq idarə etmək istəyirsənsə explicit loading uyğundur.

Lazy loading əlaqəyə toxunanda yükləyir

Bu dəfə məhsulları açıq Load əmri ilə yükləmirik. Kod `order.Items`-ə çatanda EF Core, lazımdırsa, onları avtomatik gətirir. Bu üsul **lazy loading** adlanır.

Görünən kod ilə mümkün gizli iş

```
Gorunen: order.Items siyahisini oxu.

Arxada:
    Items hele yuklenmeyib?
        Bazaya sorgu gonder.
        Gelen setirleri Items-e yerlesdir.
```

Rahat tərəfi budur ki, ayrıca yükləmə çağırışı yazmırsan. Çətin tərəfi budur ki, adi xassə oxunuşu kimi görünən sətir bazaya gedə bilər.

EF Core-da lazy loading adi entity sinfində özbaşına aktiv olmur. **Entity** bazadakı qeydi təmsil edən obyektdir. Proxy üsulunda əlavə `Microsoft.EntityFrameworkCore.Proxies` paketi və `UseLazyLoadingProxies()` ayarı istifadə edilir.

Proxy xassəyə müraciəti tutub əlavə davranış edə bilən vasitəçi obyektdir. Bu üsulda uyğun əlaqələr `virtual` olmalıdır və entity sinfindən törəmək mümkün olmalıdır. `virtual` xassənin davranışının törəmə tipdə əvəz edilə bilməsini bildirir. `ILazyLoader` adlı başqa yanaşma da var; onun texniki tələbləri fərqlidir.

Yadda saxla: lazy loading asan yazılış verir, amma baza müraciətini kodda gizlədə bilər. Növbəti səhifə bunun niyə əhəmiyyətli olduğunu göstərir.

N+1 problemini üç sifarişlə sayaq

Üç sifarişin siyahısını aldıq. Sonra hər sifarişin hələ yüklənməmiş Items əlaqəsini ayrıca oxuduq. Lazy loading aktivdirsə belə nəticə yarana bilər:

Mərhələ	Bazaya müraciət
Sifariş siyahısını gətir	1-ci sorğu
42-nin məhsullarını gətir	2-ci sorğu
43-ün məhsullarını gətir	3-cü sorğu
44-ün məhsullarını gətir	4-cü sorğu

Bir ümumi sorğu və üç əlavə sorğu var. Ümumi say `1 + N` olur. Buna **N+1 problemi** deyilir. N burada sonradan əlaqəsi ayrıca yüklənən obyektlərin sayıdır.

100 sifariş üçün eyni davranış 101 sorğu yarada bilər. Dəqiq say əvvəldən hansı məlumatın yüklənməsindən də asılıdır. Əsas problem çoxlu əlavə gediş gəlişdir: hər müraciət serverlə baza arasında vaxt və resurs tələb edir.

Explicit Load əmrlərini dövrdə 100 dəfə çağırmaq da eyni çoxsaylı müraciət problemini yarada bilər. Sadəcə əlavə iş kodda açıq görünər.

Həll: ekranın ehtiyacını əvvəlcədən müəyyən et. Uyğundursa Include ilə əlaqəni gətir, lazım olan məlumatı bir sorğuda seç və ya əlavə məlumatı toplu al. Həqiqi sorğu sayını SQL əmrlərinin qeydlərindən yoxla.

Yalnız ad və say lazımdırsa bütün obyektləri gətirmə

Ekran yalnız sifariş nömrəsi, müştəri adı və məhsul sayı göstərir. Hər məhsulun tam obyektini yükləməyə ehtiyac yoxdur. Birbaşa nəticədə lazım olan məlumatı seçə bilərik:

C#

```
var rows = await db.Orders
    .Select(o => new
    {
        o.Id,
        CustomerName = o.Customer.Name,
        ItemCount = o.Items.Count
    })
    .ToListAsync();
```

`Select` nəticənin formasını qurur. Bu seçimə **projection** deyilir. `new { ... }` burada sadə nəticə obyekti yaradır. `Count` element sayıdır. `ToListAsync` nəticələri siyahı kimi alır.

EF Core bu ifadələri SQL-ə çevirə biləndə adı və sayı bazadan lazım olan formada alırıq. Əvvəl bütün məhsulları C# yaddaşına gətirib saymaq tələb olunmur. Bu nümunədə ayrıca Include yazmaq lazım deyil.

Şəbəkəyə qaytarmaq üçün ayrıca adlandırılmış sadə nəticə tipindən də istifadə etmək olar. Belə tipə çox vaxt **DTO**, yəni məlumatı ötürmək üçün obyekt deyilir.

Seçim: “tam əlaqəli entity lazımdır?” ilə “yalnız bir neçə nəticə dəyəri lazımdır?” suallarını ayır. Siyahı ekranında projection çox vaxt daha yığcam nəticə verir.

Tracking başqa suala cavab verir

Loading “əlaqəni nə vaxt gətirim?” sualıdır. **Tracking** isə “gətirdiyim obyektdə etdiyim dəyişiklikləri izləyimmi?” sualıdır.

C#

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

order.Total = 35;
await db.SaveChangesAsync();
```

Tracking aktivdirsə EF Core obyektin əvvəlki və cari dəyərlərini nəzərə alır. `SaveChangesAsync` zamanı dəyişiklik bazaya yazıla bilər. İlkin dəyərlərin saxlanan surətinə **snapshot** deyilir.

Yalnız oxumaq istəyiriksə sorğuya `AsNoTracking()` əlavə edə bilərik. Bu halda həmin sorğunun entity-ləri əsas dəyişiklik izləyicisinə əlavə olunmur. Sonradan obyektin məbləğini dəyişmək və sadəcə SaveChanges çağırmaq həmin dəyişikliyi avtomatik yazmır.

No-tracking hər ssenaridə sabit faizlə daha sürətli deyil. Faktiki fayda nəticənin həcmi və təkrarlanan obyektlərdən asılıdır.

Eyni açarlı müştərinin nəticədə bir obyekt kimi təkrar istifadəsinə **identity resolution** deyilir. Əvvəl yüklənmiş uyğun obyektlərin əlaqələrinin avtomatik birləşdirilməsi **navigation fixup** adlanır.

Nəticə: eager ilə tracking, eager ilə no-tracking, projection və başqa kombinasiyalar mümkündür. Bunlar eyni seçim deyil.

İki kolleksiya niyə nəticəni böyüdür

Bir sifarişdə 2 məhsul və ayrıca 3 ödəniş qeydi olsun. SQL hər məhsulu hər ödənişlə eyni nəticədə birləşdirəndə həmin sifariş üçün `2 x 3 = 6` sətir yarana bilər.

JOIN əlaqəli cədvəllərin sətirlərini şərtə görə birləşdirən SQL əməliyyatıdır. Bir-birindən ayrı kolleksiyaların birləşməsi nəticəni vuraraq böyüdürsə buna **cartesian explosion** deyilir.

C#

```
var orders = await db.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSplitQuery()
    .ToListAsync();
```

`AsSplitQuery` uyğun əlaqə yükləməsini bir neçə SQL sorğusuna bölür. Böyük təkrarlanma azalır. Əvəzində əlavə baza gediş gəlişi və aralıq nəticələri yaddaşda saxlamaq lazım ola bilər. Dəqiq sorğu sayı quruluş və EF provider-indən asılıdır. **Provider** EF Core-u konkret baza sistemi ilə əlaqələndirən hissədir.

Ayrı sorğular arasında başqa istifadəçi məlumat dəyişə bilər. Hər nəticənin eyni anı əks etdirməsi tələb olunursa uyğun transaction və oxuma qaydası düşünülməlidir.

Son seçim

Əlaqə əvvəlcədən lazımdırsa eager; sonradan açıq alınacaqsa explicit; gizli alınması qəbul edilirsə lazy. Yalnız dəyərlər lazımdırsa projection seçimini düşün. Tracking-i isə dəyişiklik yazmaq ehtiyacına görə ayrıca seç.